```
import pandas as pd
import numpy as np
import os
import pickle
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, accuracy_score
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
from sklearn.metrics import classification_report, accuracy_score, confusion_matrix
import seaborn as sns
import joblib
```

```
# Mount Google Drive
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

```
!unzip -o "/content/drive/MyDrive/ML_2024-25_Sem_II/WESAD.zip" -d /content/
```

```
  inflating: /content/WESAD/S14/S14_readme.txt
  inflating: /content/WESAD/S14/S14_respiban.txt
  inflating: /content/WESAD/S15/S15.pkl
  inflating: /content/WESAD/S15/S15_E4_Data.zip
  inflating: /content/WESAD/S15/S15_quest.csv
  inflating: /content/WESAD/S15/S15_readme.txt
  inflating: /content/WESAD/S15/S15_respiban.txt
  inflating: /content/WESAD/S16/S16.pkl
  inflating: /content/WESAD/S16/S16_E4_Data.zip
  inflating: /content/WESAD/S16/S16_quest.csv
  inflating: /content/WESAD/S16/S16_readme.txt
  inflating: /content/WESAD/S16/S16_respiban.txt
  inflating: /content/WESAD/S17/S17.pkl
  inflating: /content/WESAD/S17/S17_E4_Data.zip
  inflating: /content/WESAD/S17/S17_quest.csv
  inflating: /content/WESAD/S17/S17_readme.txt
  inflating: /content/WESAD/S17/S17_respiban.txt
  inflating: /content/WESAD/S2/S2.pkl
  inflating: /content/WESAD/S2/S2_E4_Data.zip
  inflating: /content/WESAD/S2/S2_quest.csv
  inflating: /content/WESAD/S2/S2_readme.txt
  inflating: /content/WESAD/S2/S2_respiban.txt
  inflating: /content/WESAD/S3/S3.pkl
  inflating: /content/WESAD/S3/S3_E4_Data.zip
  inflating: /content/WESAD/S3/S3_quest.csv
  inflating: /content/WESAD/S3/S3_readme.txt
  inflating: /content/WESAD/S3/S3_respiban.txt
  inflating: /content/WESAD/S4/S4.pkl
  inflating: /content/WESAD/S4/S4_E4_Data.zip
  inflating: /content/WESAD/S4/S4_quest.csv
  inflating: /content/WESAD/S4/S4_readme.txt
  inflating: /content/WESAD/S4/S4_respiban.txt
  inflating: /content/WESAD/S5/S5.pkl
  inflating: /content/WESAD/S5/S5_E4_Data.zip
  inflating: /content/WESAD/S5/S5_quest.csv
  inflating: /content/WESAD/S5/S5_readme.txt
  inflating: /content/WESAD/S5/S5_respiban.txt
  inflating: /content/WESAD/S6/S6.pkl
  inflating: /content/WESAD/S6/S6_E4_Data.zip
  inflating: /content/WESAD/S6/S6_quest.csv
  inflating: /content/WESAD/S6/S6_readme.txt
  inflating: /content/WESAD/S6/S6_respiban.txt
  inflating: /content/WESAD/S7/S7.pkl
  inflating: /content/WESAD/S7/S7_E4_Data.zip
  inflating: /content/WESAD/S7/S7_quest.csv
  inflating: /content/WESAD/S7/S7_readme.txt
  inflating: /content/WESAD/S7/S7_respiban.txt
  inflating: /content/WESAD/S8/S8.pkl
  inflating: /content/WESAD/S8/S8_E4_Data.zip
  inflating: /content/WESAD/S8/S8_quest.csv
  inflating: /content/WESAD/S8/S8_readme.txt
  inflating: /content/WESAD/S8/S8_respiban.txt
  inflating: /content/WESAD/S9/S9.pkl
  inflating: /content/WESAD/S9/S9_E4_Data.zip
  inflating: /content/WESAD/S9/S9_quest.csv
  inflating: /content/WESAD/S9/S9_readme.txt
  inflating: /content/WESAD/S9/S9_respiban.txt
  inflating: /content/WESAD/wesad_readme.pdf
```

```
!ls /content/WESAD/S2/
```

```
S2_E4_Data.zip  S2.pkl  S2_quest.csv  S2_readme.txt  S2_respiban.txt
```

```
import pickle

data_path = "/content/WESAD/S2/S2.pkl"

with open(data_path, 'rb') as f:
    data = pickle.load(f, encoding='latin1')

print("✅ Data loaded successfully!")
```

```
✅ Data loaded successfully!
```

```
eda = np.array(data['signal']['wrist']['EDA'])
temp = np.array(data['signal']['wrist']['TEMP'])
acc = np.array(data['signal']['wrist']['ACC'])
labels = np.array(data['label'])


# Make sure all arrays are the same length as labels
min_len = min(len(labels), len(eda), len(temp), len(acc))
eda = eda[:min_len]
temp = temp[:min_len]
acc = acc[:min_len]
labels = labels[:min_len]

# Filter for valid labels (0=baseline, 1=stress, 2=amusement)
valid_mask = (labels == 0) | (labels == 1) | (labels == 2)
eda = eda[valid_mask]
temp = temp[valid_mask]
acc = acc[valid_mask]
labels = labels[valid_mask]



def extract_features(eda, temp, acc):


    features = []
    features.append(np.array(feature_vector, dtype=np.float64))
    for i in range(0, len(eda), 128):
        if i + 128 <= len(eda):
            eda_window = eda[i:i+128]
            temp_window = temp[i:i+128]
            acc_window = acc[i:i+128, :]

            feature_vector = [
                np.mean(eda_window), np.std(eda_window), np.max(eda_window), np.min(eda_window),
                np.mean(temp_window), np.std(temp_window), np.max(temp_window), np.min(temp_window),
                np.mean(acc_window[:,0]), np.std(acc_window[:,0]),
                np.mean(acc_window[:,1]), np.std(acc_window[:,1]),
                np.mean(acc_window[:,2]), np.std(acc_window[:,2])


            ]
            features.append(feature_vector)
    return np.array(features)



def align_labels(labels, window_size=128):
    new_labels = []
    for i in range(0, len(labels), window_size):
        if i + window_size <= len(labels):
            window = labels[i:i+window_size]
            new_labels.append(np.bincount(window).argmax())
    return np.array(new_labels)



X = extract_features(eda, temp, acc)
y = align_labels(labels)
```

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, stratify=y, random_state=42)
```

```
model = RandomForestClassifier(n_estimators=200, max_depth=10, random_state=42)
model.fit(X_train, y_train)
```

```
RandomForestClassifier(max_depth=10, n_estimators=200, random_state=42)
```

```
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

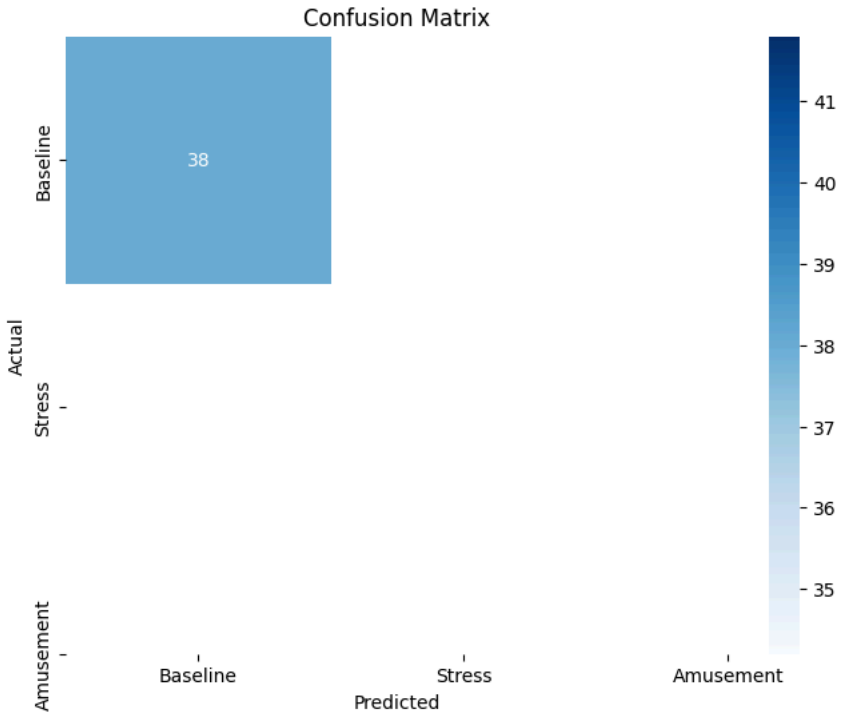
```
Accuracy: 1.0
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        38

    accuracy                           1.00        38
   macro avg       1.00      1.00      1.00        38
weighted avg       1.00      1.00      1.00        38
```

```
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=["Baseline", "Stress", "Amusement"], yticklabels=["Baseline", "Stress", '
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```

```
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:407: UserWarning: A single label was found in 'y_true' ar
  warnings.warn(
```




```
joblib.dump(model, '/content/drive/MyDrive/ML_2024-25_Sem_II/stress_model.pkl')
joblib.dump(scaler, '/content/drive/MyDrive/ML_2024-25_Sem_II/stress_scaler.pkl')
```

```
['/content/drive/MyDrive/ML_2024-25_Sem_II/stress_scaler.pkl']
```

```
print("Feature Importances:\n", importances)
print("Sum of Importances:", np.sum(importances))
```

```
Feature Importances:
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
```

```
from sklearn.ensemble import ExtraTreesClassifier
model = ExtraTreesClassifier(n_estimators=200, random_state=42)



print(importances)
```

```
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
```

```
feature_names = [
    'EDA_Mean', 'EDA_Std', 'EDA_Max', 'EDA_Min',
    'TEMP_Mean', 'TEMP_Std', 'TEMP_Max', 'TEMP_Min',
    'ACC_X_Mean', 'ACC_X_Std',
    'ACC_Y_Mean', 'ACC_Y_Std',
    'ACC_Z_Mean', 'ACC_Z_Std'
]
importances = model.feature_importances_
indices = np.argsort(importances)[::-1]

plt.figure(figsize=(10,6))
plt.title("Feature Importances")
plt.bar(range(len(importances)), importances[indices], align="center")
plt.xticks(range(len(importances)), [feature_names[i] for i in indices], rotation=45)
plt.tight_layout()
plt.show()
```